Protecting the Right Weights on a Noisy Hybrid SLC/MLC ReRAM Chip

Author One
Department
Institution
City, Country
email@example.com

Author Two
Department
Institution
City, Country
email@example.com

***Abstract*—Analog resistive-RAM (ReRAM) processing-in-memory computes matrix–vector products inside the memory array and removes most of the data movement that dominates transformer inference. Its weakness is that a cell cannot be programmed exactly. A single-level cell (SLC) stores one bit and tolerates this error, while a multi-level cell (MLC) stores two bits and is denser but far less robust. A hybrid chip keeps the important weights in SLC and the rest in MLC, so the central question is which weights to protect. We study this question for DistilBERT and BERT-Base on SST-2 with a cell-level crossbar simulator that models 8-bit weights, differential pairs, bit-slicing, programming noise, stuck-at faults, read noise, a fixed ADC and IR drop. After a singular value decomposition (SVD) of every weight matrix, we compare six placements of the SVD components: all-SLC, all-MLC, random, largest singular value (magnitude), a HyFlexPIM-style loss gradient, and a Fisher-information estimate of the damage each component would suffer in MLC. Storing everything in MLC fails at realistic noise, and BERT-Base falls from about 90% to near chance. Protecting a small share of components recovers almost all of the loss, but only if the right components are chosen. Ranking by singular value matches the best placement we could construct, whereas the gradient placement loses 3–12 accuracy points at high noise. Stuck-at cells are the one failure that placement does not fix: all-MLC fails at 0.1% stuck cells and all-SLC at 1%.**

***Index Terms*—processing-in-memory, ReRAM, multi-level cell, transformer, singular value decomposition, fault tolerance**

I. Introduction

Transformer models [2] are expensive to run, mainly because billions of weights must be moved between memory and the processor. Processing-in-memory (PIM) removes this movement by computing where the weights are stored. Analog ReRAM crossbars are an efficient form of PIM, but a cell cannot be programmed exactly, so every stored weight carries error. Dense multi-level cells (MLC) suffer most from this error, and reliable single-level cells (SLC) are costly. A hybrid chip therefore keeps only the most important weights in SLC, and the open question is which weights these are.

This paper studies that question for SVD-factorised transformers on a simulated hybrid SLC/MLC ReRAM chip. Section II gives the background, including the HyFlexPIM method [1] and our contributions. The remaining sections will describe the simulator, the experiments and the results.

II. Background

A. Analog ReRAM Crossbars

A ReRAM crossbar computes a matrix–vector product inside memory. Each weight is stored as the conductance of resistive cells. A voltage proportional to each input is applied to a row, and the currents on each column add up to the output. This avoids most of the data movement of conventional hardware, and it computes a whole column in one step.

The cost is accuracy. A cell cannot be programmed to an exact conductance, and cells can also be stuck at a fixed conductance. A single-level cell (SLC) stores one bit in two well-separated levels and tolerates considerable error. A multi-level cell (MLC) stores two bits in four closely spaced levels. It needs fewer cells per weight, which gives higher density, throughput and energy efficiency, but the same absolute conductance error is a larger error in the stored value. Transformers are sensitive to such error, and running BERT entirely on two-bit cells can cost tens of accuracy points [1].

B. HyFlexPIM

HyFlexPIM [1] is a hybrid SLC/MLC mixed-signal PIM accelerator. A single analog PIM module can switch between SLC and MLC operation with under 1% area and energy overhead, so critical weights can be stored in SLC and the remaining weights in the denser MLC. Brute-force mapping on such a chip gives little benefit, because too few weights are error-tolerant enough for MLC. HyFlexPIM therefore adds an algorithmic step called *gradient redistribution*:

1) Factorise each weight matrix with singular value decomposition (SVD) into components ordered by singular value.
2) Truncate the smallest components to remove redundancy.
3) Fine-tune the truncated model, which concentrates the loss gradient in a small subset of components.
4) Rank the components by the absolute gradient of the loss with respect to their singular value.
5) Store the top $k\%$ in SLC and the remaining components in MLC.

All of these steps run in software before deployment, and the hardware only receives the final truncated matrices. HyFlex-

PIM models MLC error by multiplying each weight by $(1+\eta)$ with Gaussian η, with the variance chosen to match a bit error rate of about 4.04% measured on fabricated MLC RRAM. It reports that protecting 5–30% of components in SLC keeps most GLUE tasks within 1% of the noise-free baseline, and that gradient-based selection outperforms magnitude- and rank-based selection on MRPC and CoLA with BERT-Base.

C. Open Questions and Our Contributions

The evidence for gradient-based selection covers a small number of tasks and a single noise model, and the model has no stuck-at cells. It is unclear whether the rule holds under other error models, whether a cheaper rule performs as well, and how stuck-at cells affect each placement. Our contributions address these points:

- **A crossbar simulator.** A cell-level PyTorch model of SVD-factorised transformers on SLC and MLC crossbars, covering weight quantisation, differential pairs, bit-slicing, programming noise, stuck-at faults, read noise, a fixed ADC and IR drop. All placements are evaluated on the same simulated chips.
- **Criticality-aware component placement.** A comparison of six policies for choosing which SVD components are stored in SLC: all-SLC, all-MLC, random, singular-value magnitude, the HyFlexPIM gradient, and a Fisher-information estimate of the damage each component would suffer in MLC.
- **Variation with SLC share.** How the accuracy of each placement policy changes as the share of components stored in SLC increases.
- **Fault analysis.** An evaluation of how programming noise and stuck-at cells, which HyFlexPIM does not model, affect each placement policy.
- **Fault-aware mapping.** A post-test remapping that uses the freedom to store SVD components in any order: once a chip's stuck cells are known, each component is assigned to the physical slot where it would raise the loss least, at no hardware cost.

We study DistilBERT and BERT-Base on the SST-2 task of GLUE.

III. Methodology

Our pipeline has three stages: SVD factorisation of the transformer, a cell-level crossbar simulation, and the placement of the SVD components on SLC and MLC cells. This section describes the first stage.

A. SVD of the Transformer

1) Models and dataset: We use two encoder transformers, DistilBERT [3] and BERT-Base [2], and the SST-2 sentiment task of GLUE [4]. SST-2 is a binary classification task. It has 67,349 training sentences and 872 validation sentences, and all accuracies in this paper are measured on the validation set. Both models start from public checkpoints that are already fine-tuned on SST-2 (`distilbert-base-uncased-finetuned-sst-2-english` and `textattack/bert-base-uncased-SST-2`). Sentences are truncated to 128 tokens.

TABLE I
SST-2 validation accuracy (%) of the SVD-factorised models after fine-tuning.

Model	SVD + fine-tune
DistilBERT	89.11
BERT-Base	90.14

2) Factorisation: Following HyFlexPIM, we factorise every linear layer of the encoder blocks (the query, key, value and output projections and the two feed-forward layers) as

$$W \approx U\,\mathrm{diag}(\sigma)\,V^{\top}, \qquad W \in \mathbb{R}^{m\times n}, \tag{1}$$

which gives 36 factorised layers for DistilBERT and 72 for BERT-Base. We keep the $k = \lfloor mn/(m+n) \rfloor$ largest singular values (for example $k = 384$ for a 768×768 layer), the largest rank for which the two factors U and $\sigma V^{\top}$ together hold no more parameters than W. On the chip each layer then becomes two crossbars, $x \to [\sigma V^{\top}] \to [U] \to y$, and component r consists of row r of $\sigma V^{\top}$ together with column r of U. This gives 16,584 components for DistilBERT and 33,168 for BERT-Base.

3) Recovery fine-tuning: Truncating to rank k removes information, so we fine-tune the factorised model on the SST-2 training set to recover the lost accuracy. We use AdamW with a learning rate of 2×10^{-5} decayed linearly to zero, batch size 32, 8 epochs, mixed-precision (fp16) autocast and seed 0. Table I shows the validation accuracy of the SVD-factorised models after fine-tuning.

All later experiments use these fine-tuned factorised models as their noise-free reference, so the accuracies in the tables that follow should be read against 89.11% (DistilBERT) and 90.14% (BERT-Base).

B. INT8 Quantisation

A crossbar stores integers, so every weight and every input is quantised to 8 bits before it reaches the array. Weights use a symmetric 8-bit scale per output row, and inputs use an 8-bit scale per tensor. In the simulator, an ideal crossbar with no noise or faults reproduces an INT8 matrix product exactly (relative error below 10^{-4}). The noise-free all-SLC chip therefore measures the accuracy of the INT8 model, and every difference between a noisy chip and this value is caused by the cell errors alone.

Table II compares floating-point (FP32) and INT8 accuracy for the original checkpoints and for the fine-tuned SVD models.

INT8 quantisation costs 0.6–0.7 points for the original models and 0.1–0.2 points for the SVD models. It is therefore a small effect compared with the 1.9–2.3 points lost to the factorisation, and it applies equally to every placement we compare.

TABLE II
SST-2 VALIDATION ACCURACY (%) IN FP32 AND INT8, WITH NO HARDWARE ERROR.

	Original		SVD + fine-tune	
Model	FP32	INT8	FP32	INT8
DistilBERT	91.06	90.37	89.11	88.99
BERT-Base	92.43	91.86	90.14	89.91

C. Crossbar Simulator

We simulate the chip cell by cell in PyTorch. Each factorised layer becomes two crossbars per tier, one holding $\sigma V^\top$ and one holding U, and every weight is stored in individual cells that can be perturbed.

1) Mapping a weight to cells: A weight matrix W is quantised to signed 8-bit integers $q = \mathrm{round}(W/s_i)$ with one scale $s_i = \max_j |W_{ij}|/127$ per output row i. Because a conductance cannot be negative, each integer is stored as a differential pair, $q = q^+ - q^-$ with $q^\pm \geq 0$. Each magnitude is split into base-L digits, $L = 2^b$, where b is the number of bits per cell:

$$d_s = \lfloor m/L^s \rfloor \bmod L, \qquad m \in \{q^+, q^-\}. \tag{2}$$

An SLC cell ($b = 1$, $L = 2$) needs 7 slices and an MLC cell ($b = 2$, $L = 4$) needs 4. Digit d is programmed as the conductance

$$g = g_{\min} + d\,\Delta, \qquad \Delta = \frac{1 - g_{\min}}{L - 1}, \tag{3}$$

with conductances normalised so that $G_{\max} = 1$ and $g_{\min} = 1/100$ (an on/off ratio of 100). The levels of an MLC cell are therefore three times closer together than those of an SLC cell. At read-out the digit is recovered as $\hat{d}_s = (g_s^+ - g_s^-)/\Delta$, and the slices are combined by shift-and-add, $\hat{W}_{i\cdot} = s_i \sum_s L^s \hat{d}_s$.

2) Error models: Each error is applied to the programmed conductances, and the same random draw is used by every placement that is evaluated on the same chip.

- **Programming noise.** Every cell receives $g \leftarrow \max\big(0,\ g + \varsigma\,\mathcal{N}(0,1)\big)$, where ς is a fraction of $G_{\max}$. Since Δ is smaller for MLC, the same ς corresponds to a larger digit error in MLC. In some experiments ς is scaled by a factor that depends on the stored level.
- **Stuck-at faults.** A cell is stuck with probability p. A fraction $a = 0.5$ of stuck cells is stuck at $G_{\max}$ (SA1) and the rest at $g_{\min}$ (SA0), whatever value was programmed. Faults are drawn independently per cell by default. We also model clusters of 3×3 cells and failed bit lines.
- **Read noise.** Gaussian noise of standard deviation proportional to $G_{\max}$ is added fresh at every read, in contrast to programming noise, which is fixed once the chip is programmed.
- **IR drop.** Wire resistance reduces each conductance to $g/(1 + r\,g\,n)$, where n counts the wire segments between the cell and its driver and sense amplifier, and r is the segment resistance relative to R_{on}.

3) Read-out and chip definition: The input is quantised to 8 bits with one scale per tensor, as described above. Arrays are 128×128 tiles. By default the read-out is ideal. In the ADC experiments each slice of each tile is digitised separately by an ADC whose range is fixed once on a noise-free chip over 64 calibration sentences, with inputs applied bit-serially in sign-magnitude form. A *chip* is one random draw of all programming noise and faults, identified by a seed. All placements are evaluated on the same set of chips, so the difference between two placements on a given chip does not come from different random draws. All simulations run in FP32.

D. Placement Policies

A placement decides which SVD components are stored in the SLC tier and which in the MLC tier. The factorised model is the same for every placement. Given an SLC share f, each layer with k components stores the $\lceil fk \rceil$ highest-scoring components in SLC and the remaining ones in MLC. Selection is done within each layer, so every layer keeps the same SLC share. We compare six policies.

- **All-SLC** and **all-MLC** store every component in one tier. They are the most reliable and the densest chip, and they bound the other policies.
- **Random** selects a random set of components. It is a control that shows what a placement without information achieves. The set is fixed, and only the cell errors differ between chips.
- **Magnitude** scores component r by its singular value σ_r. The largest singular values carry most of the weight matrix.
- **Gradient** follows HyFlexPIM [1]. The score is

$$I_r = \sum_b \left| \frac{\partial L_b}{\partial \sigma_r} \right|, \tag{4}$$

  where L_b is the loss on batch b of 32 SST-2 training sentences with dropout enabled. The absolute value is taken per batch, because on a trained model the gradients of different batches point in different directions and largely cancel in a single sum.
- **Fisher** scores a component by the expected rise in loss if it is stored in MLC:

$$D_r = \sum_{e \in \mathcal{E}_r} F_e \,\mathrm{Var}(\epsilon_e), \tag{5}$$

  where $\mathcal{E}_r$ contains every stored entry of the component (row r of $\sigma V^\top$ and column r of U) and F_e is the empirical Fisher information of entry e, estimated from squared per-sentence gradients over 2,048 training sentences. A fixed amount of cell noise becomes weight noise in proportion to the 8-bit scale of the row that holds the entry, so $\mathrm{Var}(\epsilon_e)$ is proportional to the squared scale of that row. Unlike the gradient, this score covers all entries disturbed by chip noise, not only σ_r.

TABLE III
COMPONENT PLACEMENT POLICIES. ALL POLICIES STORE THE SAME FACTORISED MODEL AND DIFFER ONLY IN THE SCORE THAT SELECTS THE SLC COMPONENTS.

Policy	Score of component r	Uses loss
All-SLC	none (everything in SLC)	no
All-MLC	none (everything in MLC)	no
Random	random draw	no
Magnitude	singular value σ_r	no
Gradient	$\sum_b \lvert\partial L_b/\partial\sigma_r\rvert$	yes
Fisher	$\sum_e F_e \operatorname{Var}(\epsilon_e)$ over all stored entries	yes

The calibration sentences are the same for the gradient and Fisher scores. Both are computed once per model and are not affected by the chip noise. Table III summarises the policies.

E. Fault Analysis

HyFlexPIM models only programming noise. Real ReRAM arrays also contain stuck-at cells, which come from forming failures and endurance wear-out. A stuck cell ignores the programmed value, so its error is not small and is not centred on the target. We therefore study faults separately from noise in two experiments. Neither adds programming noise, so every accuracy loss comes from the stuck cells.

- **Fault rate.** The stuck-cell probability p is swept over 0, 0.1, 0.2, 0.5, 1, 2 and 5%, with faults drawn independently per cell in both tiers. All six placements are evaluated at an SLC share of 20% on 20 chips per point.
- **Fault pattern.** At $p = 1\%$, the stuck cells are placed in one of three ways, each with the same expected number of stuck cells: *scattered*, every cell fails independently, as in the fault-rate sweep; *clustered*, blocks of 3×3 neighbouring cells fail together, as from a local defect; and *bit line*, all 128 cells of one column of a tile fail together, as from a broken wire or sense amplifier. In all cases half of the stuck cells are stuck at the highest conductance and half at the lowest. All placements except random are evaluated at 20% SLC on 10 chips per pattern.

IV. RESULTS

A. Criticality-Aware Component Placement

We first compare the placement policies as the share of components stored in SLC grows. Tables IV and V give the SST-2 accuracy of DistilBERT and BERT-Base at programming noise ς of 3%, 5% and 7.5% of $G_{\max}$, averaged over 20 chips. All policies are evaluated on the same simulated chips. For reference, each block also lists the accuracy of the all-MLC and all-SLC chips at that noise level.

Random placement recovers little of the gap between all-MLC and all-SLC, even at 50% SLC, and its accuracy varies strongly between chips. The three informed policies recover most of it with far less SLC. At 3% noise the gap is small (2.7 points on DistilBERT, 7.1 on BERT-Base), and every informed policy reaches the all-SLC accuracy from 5–10% SLC; the policies differ by less than the chip-to-chip spread. At 5% noise, magnitude and Fisher placement are within about one point of the all-SLC chip from 5% SLC (10% for the gradient on BERT-Base). At 7.5% noise, 20% SLC leaves magnitude and Fisher about 2 points below all-SLC on DistilBERT and 1.5 points below on BERT-Base. Magnitude and Fisher differ by less than the chip-to-chip spread, so the data does not separate them. The gradient policy is the weakest of the three. It trails magnitude by 2 points at 20% SLC on DistilBERT and 3 points on BERT-Base, and by 12 points at 5% SLC on BERT-Base at 7.5% noise.

TABLE IV
DISTILBERT: SST-2 ACCURACY (%) VS. SLC SHARE (MEAN OF 20 CHIPS; BOLD = BEST).

Policy	5%	10%	20%	30%	50%
Noise $\varsigma = 3\%$ all-MLC 86.0, all-SLC 88.7					
Random	86.4	86.8	87.5	88.1	88.4
Gradient	88.4	88.5	88.6	88.8	**88.7**
Magnitude	88.4	88.6	**88.8**	**88.9**	**88.7**
Fisher	**88.5**	**88.7**	88.7	88.8	88.6
Noise $\varsigma = 5\%$ all-MLC 74.8, all-SLC 88.2					
Random	75.4	77.4	81.9	82.3	85.7
Gradient	87.3	87.3	87.7	88.1	87.8
Magnitude	**87.4**	**88.0**	88.1	**88.8**	**88.4**
Fisher	87.3	87.8	**88.4**	88.1	88.3
Noise $\varsigma = 7.5\%$ all-MLC 56.9, all-SLC 87.4					
Random	56.4	58.2	63.1	64.7	73.7
Gradient	78.0	80.7	83.1	84.8	86.0
Magnitude	80.2	**84.0**	85.2	**86.6**	86.9
Fisher	**80.8**	83.8	**85.4**	86.0	**87.0**

TABLE V
BERT-BASE: SST-2 ACCURACY (%) VS. SLC SHARE (MEAN OF 20 CHIPS; BOLD = BEST).

Policy	5%	10%	20%	30%	50%
Noise $\varsigma = 3\%$ all-MLC 82.1, all-SLC 89.2					
Random	87.0	86.9	87.6	87.3	89.0
Gradient	88.7	**90.0**	**90.0**	**89.9**	89.6
Magnitude	**89.7**	89.9	89.8	89.8	89.7
Fisher	89.6	89.8	89.8	89.8	**89.8**
Noise $\varsigma = 5\%$ all-MLC 66.8, all-SLC 88.0					
Random	72.5	73.6	76.2	77.8	83.4
Gradient	84.2	88.2	88.4	88.4	88.1
Magnitude	**88.0**	**88.6**	88.7	**88.9**	**88.9**
Fisher	87.9	**88.6**	**88.8**	**88.9**	**88.9**
Noise $\varsigma = 7.5\%$ all-MLC 53.0, all-SLC 85.7					
Random	53.6	54.2	55.2	57.5	67.3
Gradient	67.5	79.0	81.2	82.4	83.2
Magnitude	79.4	81.8	84.1	**85.5**	85.6
Fisher	**80.0**	**82.2**	**84.2**	85.1	**85.8**

B. Fault Analysis

Table VI gives the accuracy against the share of stuck cells, and Table VII the accuracy for the three fault patterns at $p =$

TABLE VI
ACCURACY (%) VS. SHARE OF STUCK CELLS, NO PROGRAMMING NOISE (20% SLC, MEAN OF 20 CHIPS).

Policy	0	0.1%	0.2%	0.5%	1%	2%
DistilBERT						
All-SLC	89.0	88.2	87.8	85.2	74.6	58.5
All-MLC	89.1	65.5	58.2	52.3	51.1	50.2
Random	88.6	76.9	63.3	51.3	51.0	49.4
Gradient	89.8	88.1	86.7	74.5	55.4	51.0
Magnitude	89.1	88.3	87.4	80.3	62.1	51.4
Fisher	89.0	88.3	87.2	80.3	62.1	51.3
BERT-Base						
All-SLC	89.9	88.3	85.6	79.0	62.7	51.4
All-MLC	90.0	58.6	52.6	50.8	51.2	49.6
Random	90.0	67.9	55.8	50.7	50.6	50.2
Gradient	89.8	89.4	87.5	71.0	52.8	50.2
Magnitude	89.7	89.6	88.6	76.8	57.9	51.0
Fisher	89.1	89.7	88.4	75.6	55.5	49.5

TABLE VII
ACCURACY (%) AT 1% STUCK CELLS FOR THREE FAULT PATTERNS (20% SLC, MEAN OF 10 CHIPS).

	DistilBERT			**BERT-Base**		
Policy	Scat.	Clust.	Line	Scat.	Clust.	Line
All-SLC	74.2	80.3	71.2	61.3	66.7	57.0
All-MLC	51.2	51.2	50.8	50.7	50.8	50.4
Gradient	55.0	56.4	52.2	52.6	51.5	51.0
Magnitude	62.5	56.1	54.7	56.4	54.0	50.4
Fisher	60.0	57.3	56.3	55.6	53.1	52.2

1%. At 5% stuck cells every placement is at chance (about 50%), so that column is omitted.

MLC is far more fragile to faults than to noise. Only 0.1% stuck cells reduce the all-MLC accuracy by 24 points on DistilBERT and 31 points on BERT-Base, whereas programming noise of 2% on every cell costs it about 1 and 3 points. Random placement fails almost as early, because it leaves important components in MLC. With magnitude or Fisher placement, the hybrid chip stays within 2 points of the fault-free accuracy up to 0.2% stuck cells, and on BERT-Base it is even more accurate than the all-SLC chip there, since SLC needs almost twice as many cells per weight. From 0.5% the hybrid chip loses 9–14 points, the gradient placement 15–19 points, and at 1% even the all-SLC chip fails. Clustered faults hurt the all-SLC chip less than scattered ones, since they damage fewer weights, while failed bit lines hurt most. At 1% stuck cells every hybrid placement is close to chance for all three patterns.

C. Fault-Aware Mapping

Placement cannot protect against faults, because a stuck cell damages whichever component is stored on it. The SVD structure, however, offers a fix at no hardware cost. Component r occupies one row of the $\sigma V^\top$ crossbar and the matching input row of the U crossbar, and the components of a layer can be stored in any order without changing its output. After a chip has been tested and its stuck cells are known, we choose which component goes into which physical slot. The cost of storing component r in slot s is its expected rise in loss,

$$C_{rs} = \sum_{e \in \mathcal{E}_r} F_e \, \mathbb{E}\big[\epsilon_{e,s}^2\big], \tag{6}$$

where $\epsilon_{e,s}$ is the weight error of entry e in slot s: the exact error caused by the slot's stuck cells plus the programming noise of the slot's tier. The Hungarian algorithm finds the assignment of lowest total cost. We test three mapped placements against the unmapped ones on the same 10 chips: *all-SLC + remap*, *magnitude + remap*, which keeps the 20% magnitude SLC set and reorders the components within each tier, and *C1*, which chooses the tier and the slot of every component in a single assignment. Table VIII gives the results.

TABLE VIII
FAULT-AWARE MAPPING: ACCURACY (%) VS. SHARE OF STUCK CELLS AND FAULT PATTERN, NO PROGRAMMING NOISE (MEAN OF 10 CHIPS).

	Scattered				1%	
Policy	0.2%	0.5%	1%	2%	Clust.	Line
DistilBERT						
All-SLC	88.1	85.4	73.7	57.9	79.6	71.1
All-SLC + remap	88.0	**87.6**	**84.8**	**62.3**	**85.6**	**81.5**
Magnitude	87.1	77.9	62.4	50.2	55.9	54.0
Magnitude + remap	**88.4**	85.1	67.0	53.3	68.5	68.0
C1	88.3	85.3	67.7	54.7	75.5	62.7
BERT-Base						
All-SLC	85.4	80.1	60.9	51.1	66.5	57.2
All-SLC + remap	89.6	83.7	**75.8**	**55.9**	**81.3**	**77.5**
Magnitude	88.5	76.6	55.9	50.1	53.7	50.1
Magnitude + remap	**90.1**	**85.6**	65.1	52.2	64.3	56.9
C1	90.0	84.3	68.3	49.4	61.5	56.8

Remapping improves every placement it is applied to. On the same chips, magnitude + remap beats magnitude at every fault rate, pattern and model. At 0.5% stuck cells it raises the accuracy from 77.9 to 85.1% on DistilBERT and from 76.6 to 85.6% on BERT-Base, and for clustered faults it gains 10–13 points. The all-SLC chip gains most at higher fault rates: at 1% stuck cells it rises from 73.7 to 84.8% on DistilBERT and from 60.9 to 75.8% on BERT-Base, and for failed bit lines by 10 and 20 points. Up to 0.2% stuck cells, the remapped hybrid chip matches the remapped all-SLC chip with about 34% fewer cells; from 0.5% the remapped all-SLC chip is clearly more robust. Choosing the tier per chip (C1) gives no reliable gain over magnitude + remap: it wins in some settings and loses in others. The gains hold when 5% programming noise is added: at 0.5% stuck cells remapping raises magnitude placement from 74.9 to 83.0% on DistilBERT and from 70.3 to 80.4% on BERT-Base.

D. IR Drop and ADC Resolution

We next test two non-idealities of the array periphery: wire resistance (IR drop) and a fixed ADC. In both experiments there is no programming noise and no fault, so the result is deterministic and each setting is evaluated on a single chip,

TABLE IX
IR DROP: ACCURACY (%) VS. WIRE RESISTANCE RATIO r (20% SLC, NO PROGRAMMING NOISE).

Policy	0	10^{-4}	$3{\cdot}10^{-4}$	10^{-3}	$3{\cdot}10^{-3}$
DistilBERT					
All-MLC	89.0	88.6	89.1	**89.9**	**77.9**
All-SLC	88.5	89.9	89.4	85.3	70.1
Magnitude	88.8	89.2	89.7	86.6	73.4
Gradient	88.6	89.6	89.6	86.9	74.7
Fisher	89.1	89.2	89.3	86.4	72.6
BERT-Base					
All-MLC	90.4	89.3	89.8	**90.6**	**79.1**
All-SLC	89.7	90.3	88.9	87.0	58.5
Magnitude	90.6	90.1	90.4	88.1	76.8
Gradient	90.3	90.5	90.5	88.6	76.4
Fisher	89.6	91.1	91.1	88.3	72.8

TABLE X
ADC: ACCURACY (%) VS. ADC RESOLUTION IN BITS (20% SLC, NO PROGRAMMING NOISE).

Policy	Ideal	8	7	6	5
DistilBERT					
All-MLC	89.0	88.3	87.8	81.7	56.5
All-SLC	88.5	89.2	88.4	88.0	71.0
Magnitude	88.8	88.6	89.0	89.7	70.6
Gradient	88.6	88.4	88.5	88.4	77.3
Fisher	89.1	88.0	88.2	88.8	70.9
BERT-Base					
All-MLC	90.4	89.6	89.2	83.0	51.1
All-SLC	89.7	89.9	90.0	89.3	51.1
Magnitude	90.6	90.4	89.3	90.0	51.1
Gradient	90.3	90.1	90.1	89.6	53.0
Fisher	89.6	90.1	89.7	90.5	51.6

with hybrid placements at 20% SLC. Differences below about one point are within the spread between placements of an ideal chip and are not meaningful.

IR drop. The wire-to-cell resistance ratio r is swept from 0 to 3×10^{-3} with no compensation. Table IX gives the results.

Up to $r = 3 \times 10^{-4}$ no placement loses accuracy. At $r = 10^{-3}$ all-MLC is unaffected while all-SLC loses 3 points, and at $r = 3 \times 10^{-3}$ all-MLC keeps 78–79% while all-SLC falls to 70% on DistilBERT and 59% on BERT-Base. IR drop therefore reverses the usual order: SLC is the more sensitive tier. The attenuation $r\,g\,n$ grows with the conductance, and SLC stores every non-zero digit at $G_{\max}$. The hybrid placements lie between the two, with no consistent difference between policies.

ADC resolution. Inputs are applied bit-serially and each slice of each tile is read by an ADC of 3 to 8 bits with a fixed range. Table X gives the results down to 5 bits; at 4 and 3 bits every placement is near chance.

A 7-bit ADC costs no accuracy for any placement. At 6 bits all-MLC loses 7 points on both models, while all-SLC and every hybrid placement remain within one point of the ideal read-out. At 5 bits every placement fails on BERT-Base, and on DistilBERT all keep only 71–77%. **MLC needs at least one more ADC bit than SLC, so the hybrid chip can run with a 6-bit ADC.** With 5% programming noise added (DistilBERT, 3 chips), the gap widens: with a 6-bit ADC all-MLC drops from 77.8% (ideal read-out) to 56.0%, while all-SLC and the hybrid placements stay at 87–89%.

REFERENCES

[1] C. E. Song, P. Bhatnagar, Z. Xia, N. S. Kim, T. S. Rosing, and M. Kang, "Hybrid SLC-MLC RRAM mixed-signal processing-in-memory architecture for transformer acceleration via gradient redistribution," in *Proc. 52nd Annu. Int. Symp. Computer Architecture (ISCA)*, Tokyo, Japan, Jun. 2025, pp. 1155–1170.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[4] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proc. ICLR*, 2019.